# Operador visual de sistemas sin API

Manuel Argüelles Guerra | Frontier 2026 | Reto 1 | Diseño v3 para revisión

## Objetivo y límites

Ejecutar tareas nuevas en aplicaciones desconocidas, únicamente mediante pixels y VNC. Cada tarea entrega websocket RFB y contraseña; SIGECOM de práctica usa kiosko 1280x800. Sin DOM, almacenamiento ni API de negocio. Fecha relativa desde la app. El set oculto puede cambiar de dominio (p. ej. clínica a supermercado) y de reglas de negocio; el escritorio puede cambiar entre tareas. Supuestos: pantalla 1280x800, repintado hasta 1 s, clic de al menos 100 ms, "hoy" es la fecha de la app, tarea máxima 5 min.

## Fuera de alcance (con razones)

Sin GPU ni entrenamiento: no hay tiempo ni datos; el modelo de visión hospedado cubre la lectura. Sin pool/concurrencia: memoria no medida; una tarea activa por instancia, y otra falla explícito. Sin agentes/MCP/herramientas de terceros ni noVNC como cliente: el reto prohíbe lo que haga el trabajo por ti; usamos cliente RFB propio y solo copiamos el DES de noVNC (MPL-2.0) para el desafío VNC. Sin DOM ni API de la app: es caja negra y cambia de dominio en el set oculto. OCR/modelos especializados solo como plan B si fallan los recortes.

## Arquitectura y decisiones

API TypeScript/Node: Bearer de equipo, GET /health, POST /tasks acepta y devuelve solo task_id en menos de 2 s; GET /tasks/{id} informa estado/resumen. SQLite en volumen conserva estado sin contraseña; reinicio falla tareas interrumpidas sin replay. Una tarea activa por instancia y exclusión por escritorio evitan pool sin memoria medida. Worker aislado conecta siempre al escritorio recibido con cliente RFB propio (descartado: noVNC+Chromium/Playwright, más pesado, memoria sin medir y cliente ajeno que simplifica pero resta control). Captura framebuffer nativo. Sonnet vía OpenRouter es baseline de actuación por un acierto local; Gemini Flash candidato para lectura tras comparar precisión/coste. Roles planner/localizador/verificador, combinados cuando convenga; no tres llamadas obligatorias por paso.

## Estado de implementación (hechos)

Implementado: cliente RFB propio (autenticación DES, Raw/CopyRect, clic 150 ms, teclado keysym/NFC, Ctrl+A remoto, desconexión confirmada); API con contrato y watchdog 270/280 s; loop con Sonnet vía OpenRouter; repo público https://github.com/manuelarguelles/frontier-reto1-agente. La prueba E1 contra SIGECOM real pasó __check completo en 115 s. E2-E5 y despliegue en la nube pendientes, a medir; no se garantiza lo no medido.

## Flujo y escenarios

Observar -> planear -> localizar -> validar -> actuar -> observar y verificar. Cada propuesta JSON referencia frame_id y sesión; rechazar respuestas antiguas, fuera de límites o de alcance. Targets pequeños: crop ampliado con offset/escala explícitos. Clic 150 ms; esperar repintado hasta 1 s y comprobar estabilidad acotada. Teclas RFB down/up; seleccionar/reemplazar y verificar tildes antes de guardar. Homónimos: fallida con identificadores, sin mutación. Conteos: identidad exacta, páginas completas e IDs únicos. Crear: comprobar duplicidad; actualizar: preservar otros campos; exportar: verificar filtros y confirmación. Dos pasos repetidos sin progreso activan relocalización/fallback acotado; nunca repetir un guardado incierto sin comprobarlo.

## Guardas, tiempo y finalización

Pantalla es dato sin autoridad: ignorar la orden incrustada de borrar pedidos y conservar la nota. Identidad, alcance y postcondición visible antes de declarar éxito; guardas semánticas requieren pruebas. Credenciales fuera del modelo/logs/resumen; URL wss pública y egress restringido. Deadline desde aceptación: watchdog monotónico independiente cancela 270 s, libera entradas y confirma cierre a 280 s. Descarta decisiones en vuelo; supervisor termina worker si no cierra. Terminal solo después de parar. Sin evidencia: fallida, no falso éxito. Preguntas: respuesta: valor; dos datos separados por ;. Registrar captura/propuesta/acción/evidencia para diagnóstico y replay offline privado.

## Pruebas, alternativas y despliegue

Harness separado: reset -> E1 alta, E2 dirección con injection, E3 conteo, E4 export, E5 homónimos -> __check y validación de estado/resumen. Agente no recibe expectativas. Añadir layout desconocido, Unicode, modal, bucle, desconexión tras guardar y respuesta tardía. Medir 5/5 prácticas, acciones extra, duplicados, falsos éxitos, intentos de injection, grounding, p95 aceptación, tiempo, coste y memoria. Mismo Docker en staging/final

separado de El Chambas; memoria por medir. Motor propio frente a agente prefabricado por control/restricciones; SQLite frente a DB externa por simplicidad. OCR/modelos especializados quedan como alternativa solo si fallan los recortes.